Public Library Consortium

Database Management System — Project Report

SQL Server Management Studio (SSMS) — Pure Backend Implementation

Database Name: PublicLibraryConsortium

Tool Used: Microsoft SQL Server Management Studio (SSMS)

Submitted By

Member	Role / Module	Name	Roll No / ID
Member 1	Books, Book_Copies, Constraints, CRUD, JOINs	**M. Waqar Altaf**	**F20242661130**
Member 2	Members, Stored Procedures, Transactions	**Moarij Irfan**	**F2024266190**
Member 3	Book_Loans, Views, Subqueries, Integration	**M. Sami Ullah Yousaf**	**F2024266119**

Date: 28 June 2026

1. Introduction

The Public Library Consortium project is a backend database management system built entirely in Microsoft SQL Server Management Studio (SSMS), without any front-end or GUI application layer. The system models the day-to-day operations of a small library consortium: cataloguing books, tracking physical copies of each book, registering members, and recording loan transactions (checkouts and returns).

The project was completed as a 3-member group assignment. The work was divided strictly along table ownership and feature type so that all three members could build and test their parts in parallel with minimal blocking dependencies, before merging everything into a single, fully integrated database on the final day.

1.1 Objectives

- Design a normalized relational schema (up to 3NF) for a library system.
- Enforce data integrity using Primary Keys, Foreign Keys, UNIQUE, and CHECK constraints.
- Demonstrate CRUD operations, INNER/LEFT JOINs, subqueries (including a correlated subquery), and aggregate functions with GROUP BY/HAVING.
- Implement business logic safely using stored procedures wrapped in transactions (BEGIN TRANSACTION / COMMIT / ROLLBACK).
- Build reporting views for overdue loans and catalog circulation statistics.
- Integrate all members' work into a single master script and validate it end-to-end.

1.2 Tools & Environment

- Database Engine: Microsoft SQL Server Express
- Client Tool: SQL Server Management Studio (SSMS)
- Database Name: PublicLibraryConsortium
- Approach: Pure T-SQL backend — no application/GUI layer

2. Database Design

The database consists of four core tables connected by foreign key relationships, forming a simple one-to-many chain from catalogue data down to transactional loan data.

Table	Purpose	Relationship
Books	Catalogue of titles (ISBN, title, author, genre, year)	1 → Many Book_Copies
Book_Copies	Each physical copy of a book, with condition & availability	1 → Many Book_Loans
Members	Registered library members and their status	1 → Many Book_Loans
Book_Loans	Checkout / return transaction records	Many-to-1 with Book_Copies and Members

Relationship chain: Books (1) → Book_Copies (Many) → Book_Loans (Many) ← Members (1)

3. Work Division Among Members

The work was split by table ownership and feature type, not by arbitrary task assignment. This kept dependencies minimal and let two of the three members work fully in parallel from Day 1.

Member	Owns	Why this grouping
M. Waqar Altaf	Books, Book_Copies + constraints + CRUD + JOINs	These two tables have no foreign keys pointing into other members' tables — they can be built first, independently, with zero dependencies.
Moarij Irfan	Members + Stored Procedures + Transactions	Members also has no dependency on Books/Copies, so it can be built in parallel with Member 1. The procedures are the business-logic layer and need both tables to exist before they can be tested — so procedures come right after.
M. Sami Ullah Yousaf	Book_Loans + Views + Subqueries/Aggregates + Integration	Book_Loans depends on both Book_Copies (Member 1) and Members (Member 2), so this work naturally starts last. Because Member 3 is the only one touching all four tables, they are also the natural person to merge everything and run final tests.

3.1.ER Diagram

The diagram below was generated directly using SQL Server Management Studio's Database Diagram tool, showing all four entities, their attributes, primary keys (key icon), and the foreign key relationships that connect them.

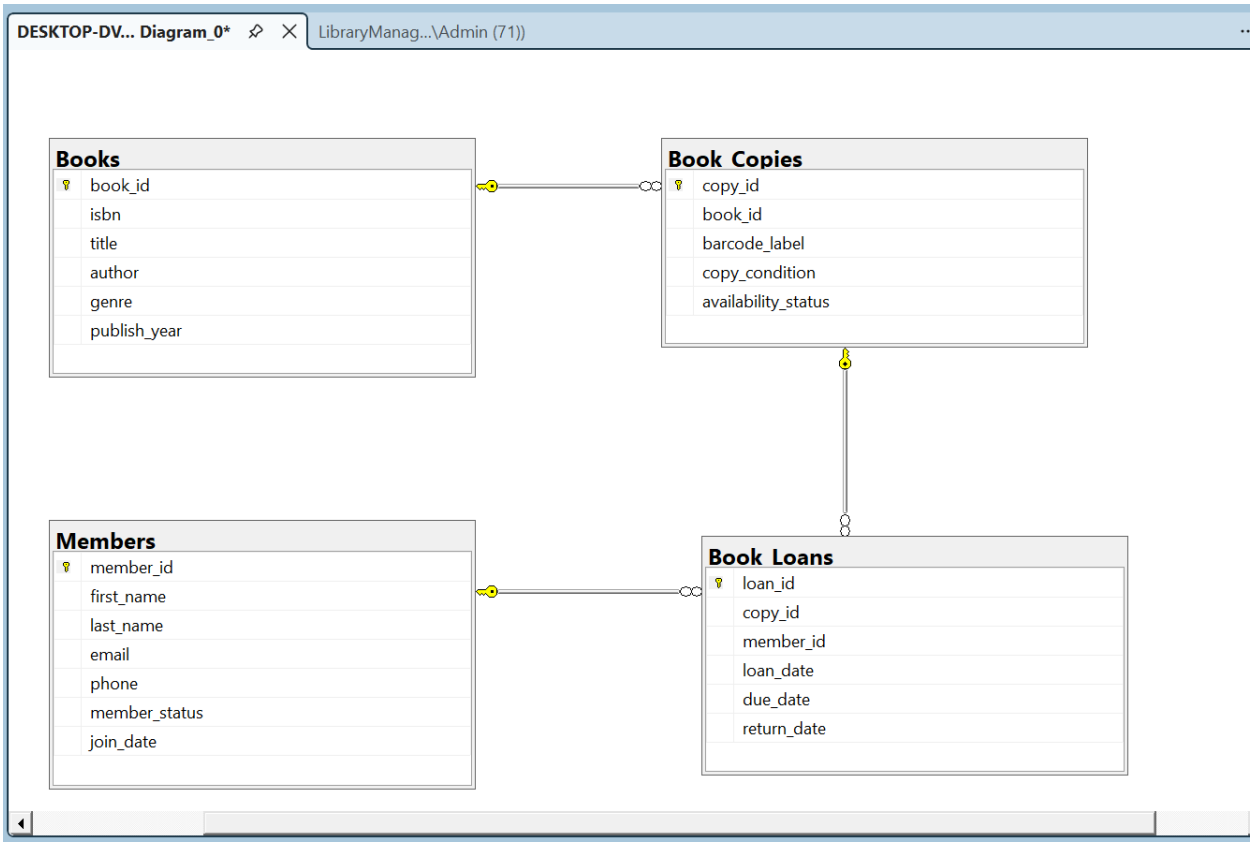


Figure 1: Entity Relationship Diagram — Books, Members, Book_Copies, Book_Loans

Relationship summary:

- Books (1) — Book_Copies (Many): one book title can have multiple physical copies.
- Book_Copies (1) — Book_Loans (Many): one physical copy can be loaned out multiple times over its lifetime (sequentially, not simultaneously).
- Members (1) — Book_Loans (Many): one member can have multiple loan records over time.

3.2 Database Schema

The schema consists of four tables, created in dependency order so that referenced parent tables exist before child tables that hold foreign keys.

3.2.1 Books Table

```
CREATE TABLE Books (
    book_id      INT IDENTITY(1,1) PRIMARY KEY,
    isbn         VARCHAR(20)  NOT NULL UNIQUE,
```

```
    title        VARCHAR(255) NOT NULL,
    author       VARCHAR(150) NOT NULL,
    genre        VARCHAR(100) NULL,
    publish_year INT          NULL
);
```

3.2.2 Members Table

```
CREATE TABLE Members (
    member_id     INT IDENTITY(1,1) PRIMARY KEY,
    first_name    VARCHAR(80)  NOT NULL,
    last_name     VARCHAR(80)  NOT NULL,
    email         VARCHAR(150) NOT NULL UNIQUE,
    phone         VARCHAR(20)  NULL,
    member_status VARCHAR(20)  NOT NULL
        CONSTRAINT CK_Members_Status CHECK (member_status IN
('Active','Suspended','Expired')),
    join_date     DATE NOT NULL DEFAULT GETDATE()
);
```

3.2.3 Book_Copies Table

```
CREATE TABLE Book_Copies (
    copy_id              INT IDENTITY(1,1) PRIMARY KEY,
    book_id              INT NOT NULL,
    barcode_label        VARCHAR(30) NOT NULL UNIQUE,
    copy_condition       VARCHAR(20) NOT NULL DEFAULT 'Good'
        CONSTRAINT CK_Copies_Condition CHECK (copy_condition IN
('New','Good','Worn','Damaged')),
    availability_status  VARCHAR(20) NOT NULL DEFAULT 'Available'
        CONSTRAINT CK_Copies_Availability CHECK (availability_status IN ('Available','On
Loan','Lost','Withdrawn')),

    CONSTRAINT FK_Copies_Books FOREIGN KEY (book_id)
        REFERENCES Books(book_id)
        ON DELETE NO ACTION
        ON UPDATE NO ACTION
);
```

3.2.4 Book_Loans Table

```
CREATE TABLE Book_Loans (
    loan_id     INT IDENTITY(1,1) PRIMARY KEY,
    copy_id     INT NOT NULL,
    member_id   INT NOT NULL,
    loan_date   DATE NOT NULL DEFAULT GETDATE(),
    due_date    DATE NOT NULL,
    return_date DATE NULL,

    CONSTRAINT FK_Loans_Copies FOREIGN KEY (copy_id)
        REFERENCES Book_Copies(copy_id)
        ON DELETE NO ACTION
        ON UPDATE NO ACTION,

    CONSTRAINT FK_Loans_Members FOREIGN KEY (member_id)
        REFERENCES Members(member_id)
        ON DELETE NO ACTION
        ON UPDATE NO ACTION
);
```

Note: ON DELETE NO ACTION is the SQL Server equivalent of the standard ANSI SQL RESTRICT clause — it blocks any delete or update that would orphan a dependent row, preserving referential integrity at all times.

3.2.5 Stored Procedures

Two transactional stored procedures encapsulate the system's core business logic.

```
CREATE PROCEDURE sp_CheckoutBookCopy
    @MemberID INT,
    @CopyID   INT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        DECLARE @MemberStatus VARCHAR(20);
        DECLARE @CopyStatus   VARCHAR(20);

        SELECT @MemberStatus = member_status FROM Members WHERE member_id = @MemberID;
        SELECT @CopyStatus   = availability_status FROM Book_Copies WHERE copy_id = @CopyID;

        IF @MemberStatus IS NULL
            RAISERROR('Member not found.', 16, 1);
        IF @MemberStatus <> 'Active'
            RAISERROR('Checkout denied: member is not Active.', 16, 1);
        IF @CopyStatus IS NULL
            RAISERROR('Copy not found.', 16, 1);
        IF @CopyStatus <> 'Available'
            RAISERROR('Checkout denied: copy is not Available.', 16, 1);

        INSERT INTO Book_Loans (copy_id, member_id, loan_date, due_date)
        VALUES (@CopyID, @MemberID, GETDATE(), DATEADD(DAY, 14, GETDATE()));

        UPDATE Book_Copies SET availability_status = 'On Loan' WHERE copy_id = @CopyID;

        COMMIT TRANSACTION;
        PRINT 'Checkout successful.';
    END TRY
    BEGIN CATCH
        IF XACT_STATE() <> 0 ROLLBACK TRANSACTION;
        PRINT 'Checkout failed: ' + ERROR_MESSAGE();
    END CATCH
END;

CREATE PROCEDURE sp_ReturnBookCopy
    @LoanID INT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;
    BEGIN TRY
        BEGIN TRANSACTION;
        DECLARE @CopyID INT;
        DECLARE @AlreadyReturned DATE;

        SELECT @CopyID = copy_id, @AlreadyReturned = return_date
        FROM Book_Loans WHERE loan_id = @LoanID;
```

```
        IF @CopyID IS NULL
            RAISERROR('Loan record not found.', 16, 1);
        IF @AlreadyReturned IS NOT NULL
            RAISERROR('This loan has already been returned.', 16, 1);

        UPDATE Book_Loans SET return_date = GETDATE() WHERE loan_id = @LoanID;
        UPDATE Book_Copies SET availability_status = 'Available' WHERE copy_id = @CopyID;

        COMMIT TRANSACTION;
        PRINT 'Return processed successfully.';
    END TRY
    BEGIN CATCH
        IF XACT_STATE() <> 0 ROLLBACK TRANSACTION;
        PRINT 'Return failed: ' + ERROR_MESSAGE();
    END CATCH
END;
```

3.2.6 Reporting Views

```
CREATE VIEW vw_OverdueLoanFulfillment AS
SELECT
    b.title AS book_title,
    bc.barcode_label,
    m.first_name + ' ' + m.last_name AS borrower_name,
    bl.due_date,
    DATEDIFF(DAY, bl.due_date, GETDATE()) AS days_overdue
FROM Book_Loans bl
INNER JOIN Book_Copies bc ON bl.copy_id = bc.copy_id
INNER JOIN Books b        ON bc.book_id = b.book_id
INNER JOIN Members m      ON bl.member_id = m.member_id
WHERE bl.return_date IS NULL
  AND bl.due_date < CAST(GETDATE() AS DATE);

CREATE VIEW vw_CatalogCirculationSummary AS
SELECT
    b.book_id,
    b.title,
    COUNT(DISTINCT bc.copy_id) AS total_copies_owned,
    COUNT(DISTINCT CASE WHEN bl.return_date IS NULL THEN bl.loan_id END) AS active_loans,
    COUNT(bl.loan_id) AS lifetime_total_loans
FROM Books b
LEFT JOIN Book_Copies bc ON b.book_id = bc.book_id
LEFT JOIN Book_Loans bl  ON bc.copy_id = bl.copy_id
GROUP BY b.book_id, b.title;
```

4. Member 1 — Schema Foundation, Constraints, CRUD, JOINs

M. Waqar Altaf was responsible for the two foundation tables, Books and Book_Copies. These tables have no foreign keys pointing into the other members' tables, so they could be designed and populated first, completely independently.

4.1 Tasks Completed

- Created the Books table with PRIMARY KEY, UNIQUE (isbn), and NOT NULL constraints.
- Created the Book_Copies table with a FOREIGN KEY to Books and two CHECK constraints (copy_condition, availability_status).
- Inserted 30 seed rows into Books and 30 seed rows into Book_Copies.
- Wrote CRUD demonstration queries (INSERT / SELECT / UPDATE / DELETE).
- Wrote INNER JOIN and LEFT JOIN queries across Books, Book_Copies, Members, and Book_Loans.
- Ran the duplicate-ISBN UNIQUE constraint test and the delete-blocked-by-FK safeguard test.

4.2 Books Table Creation

The Books table stores the catalogue. book_id is a surrogate primary key (IDENTITY), isbn is forced UNIQUE so no two books can share an ISBN, and title/author are mandatory fields.

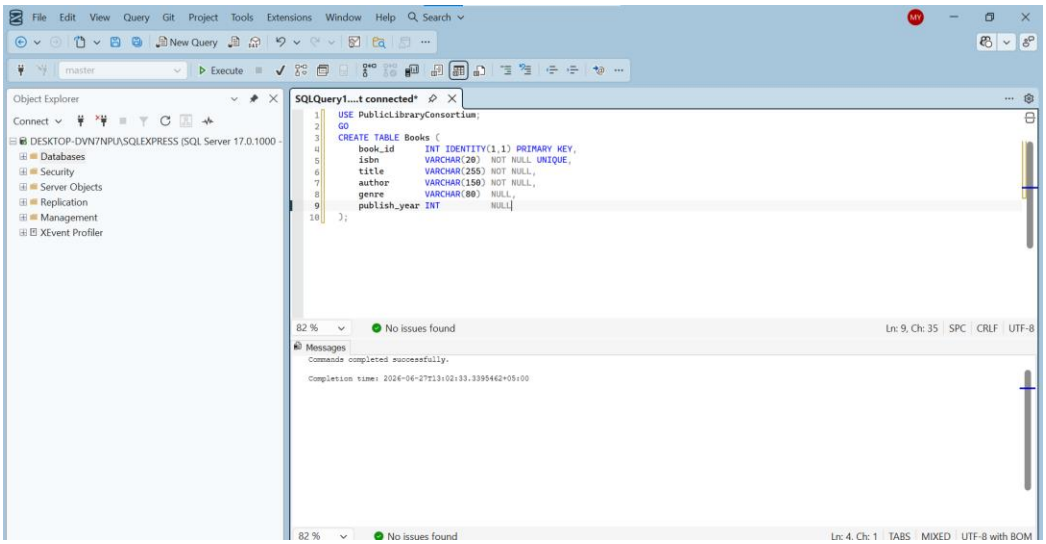


Figure 4.1 — CREATE TABLE Books, executed successfully in SSMS.

4.3 Book_Copies Table Creation

Each row in Book_Copies represents one physical copy of a book. The foreign key to Books uses ON DELETE NO ACTION, meaning SQL Server will refuse to delete a Book that still has copies referencing it — this is verified later in the constraint testing section.

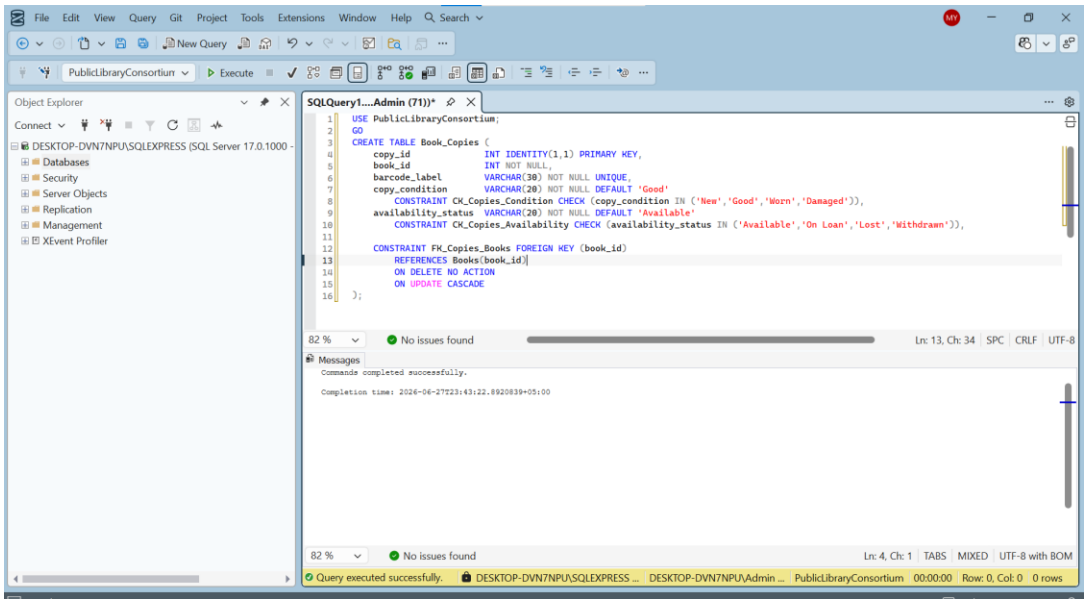


Figure 4.2 — CREATE TABLE Book_Copies with FK, CHECK constraints on copy_condition and availability_status.

4.4 Seed Data — Books (30 rows)

30 books spanning a wide mix of genres (Fiction, Fantasy, Science Fiction, Romance, Horror, Non-Fiction, Classic) were inserted so that downstream JOINs, subqueries, and GROUP BY queries would have meaningful, varied data to work with.

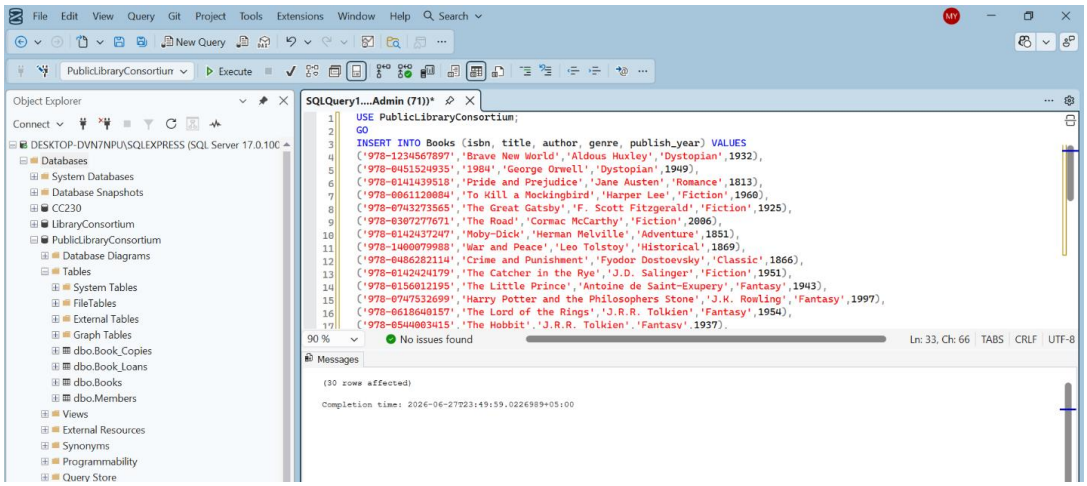


Figure 4.3 — 30 rows successfully inserted into Books.

4.5 Seed Data — Book_Copies (30 rows)

30 physical copies were inserted across the 30 books, with several books receiving 2 copies and a few copies pre-marked as 'On Loan' so that Member 3's Book_Loans seed data would have real, matching copies to reference.

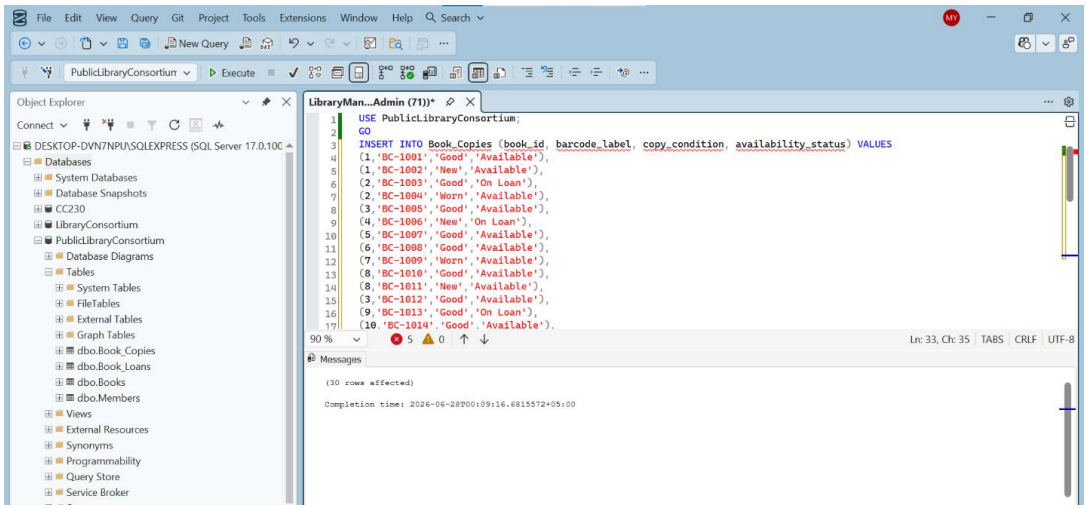


Figure 4.4 — 30 rows successfully inserted into Book_Copies.

4.6 INNER JOIN — Books with Book_Copies

This query lists every physical copy alongside its book title and current availability status, using an INNER JOIN, which only returns rows where a matching book_id exists on both sides.

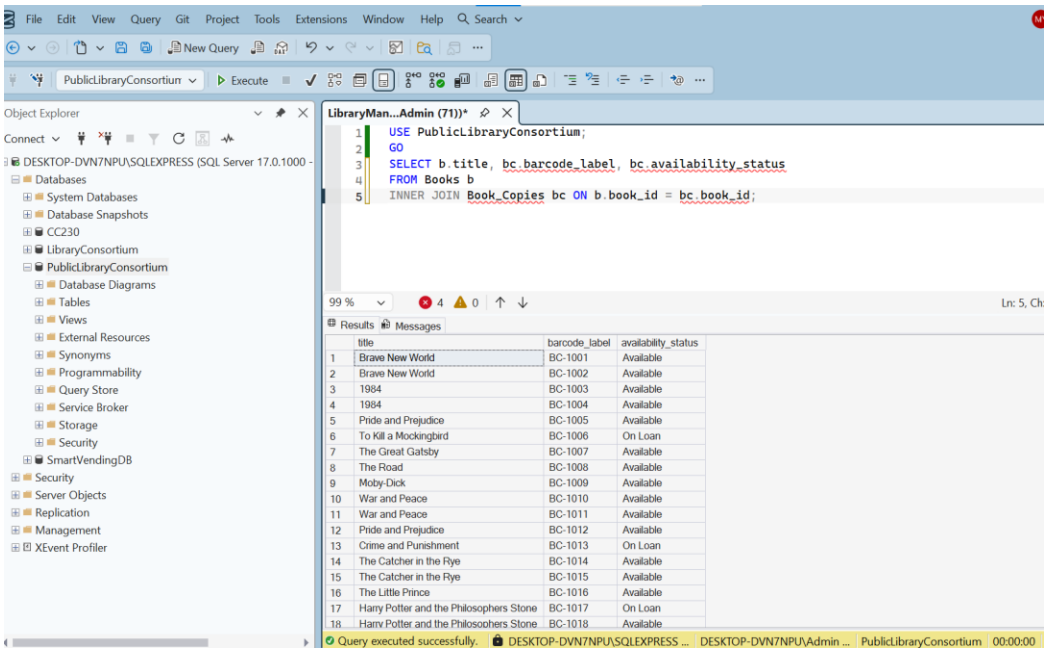


Figure 4.5 — INNER JOIN result: every copy shown with its book title and status.

4.7 LEFT JOIN — Books, Copies, Loans, and Members

This query shows every copy and, where applicable, who currently has it on loan. A LEFT JOIN is used for the loan and member side because most copies are not currently on loan — LEFT JOIN keeps those rows and simply shows NULL for the borrower, instead of hiding the row the way an INNER JOIN would.

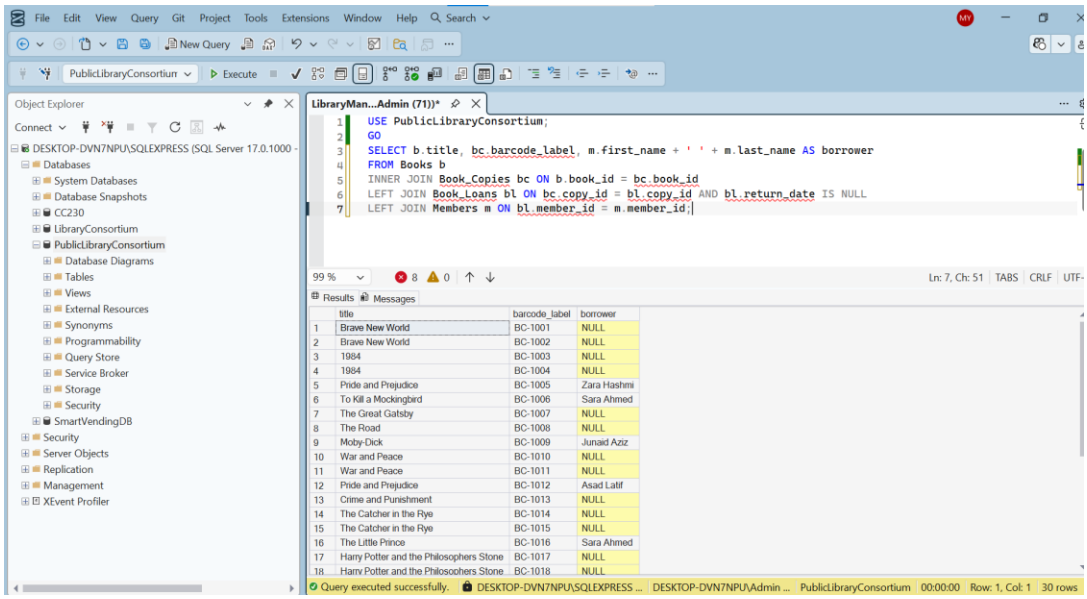


Figure 4.6 — LEFT JOIN result: copies with borrower name where applicable, NULL otherwise.

4.8 Constraint Test — Duplicate ISBN (UNIQUE Violation)

To prove the UNIQUE constraint on isbn actually works, an INSERT was deliberately attempted using an ISBN that already existed in the table. SQL Server correctly rejected the statement.

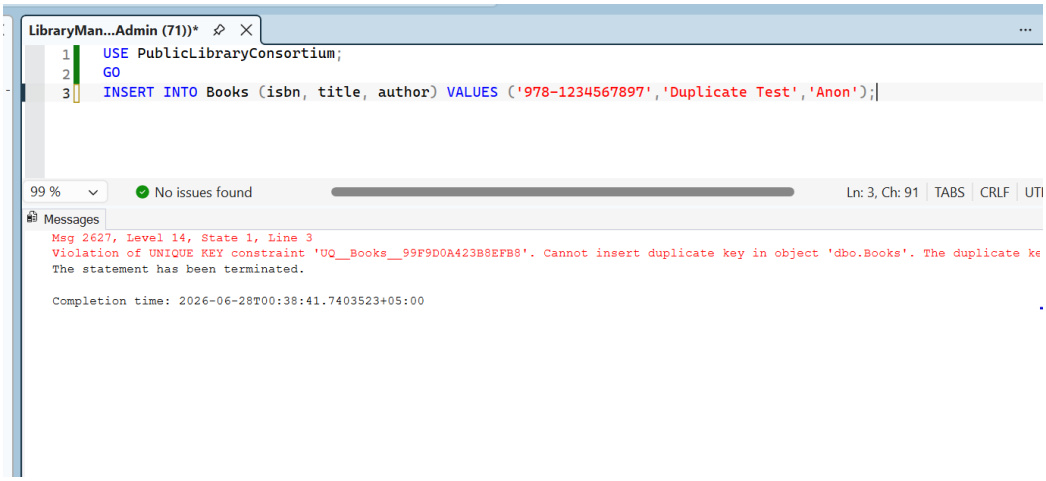


Figure 4.7 — Duplicate ISBN insert rejected with a UNIQUE KEY constraint violation.

4.9 Constraint Test — Delete Blocked by Foreign Key

To prove referential integrity, a DELETE was deliberately attempted on a Book that still had copies (and loans) referencing it. The FOREIGN KEY constraint correctly blocked the delete.

Figure 4.8 — DELETE statement blocked by FK_Copies_Books, proving referential integrity is enforced.

5. Member 2 — Members Table, Stored Procedures, Transactions

Moarij Irfan owned the Members table and the entire business-logic layer: the two stored procedures that handle checking out and returning books, both wrapped safely in SQL transactions.

5.1 Tasks Completed

- Created the Members table with a CHECK constraint restricting member_status to Active, Suspended, or Expired.
- Inserted 30 seed members with a realistic mix of all three statuses.
- Wrote sp_CheckoutBookCopy, which validates the member and copy before creating a loan.
- Wrote sp_ReturnBookCopy, which validates the loan before marking it returned.
- Tested both procedures with valid and deliberately invalid inputs.

5.2 Members Table Creation

member_status is restricted to exactly three values using a CHECK constraint. This is the column the checkout procedure relies on to decide whether a member is allowed to borrow a book.

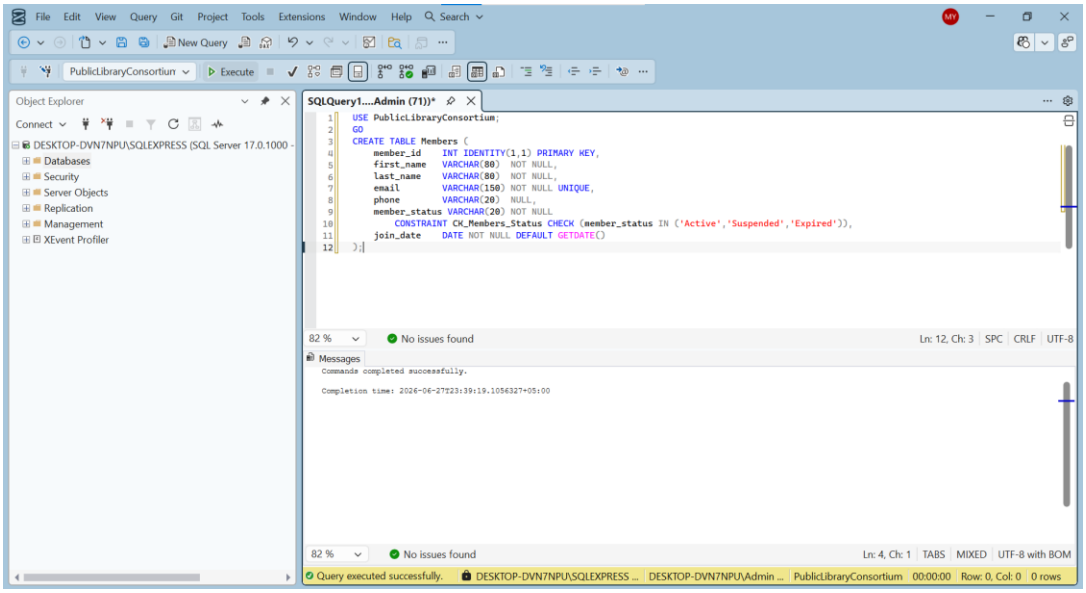


Figure 5.1 — CREATE TABLE Members with CK_Members_Status CHECK constraint.

5.3 Seed Data — Members (30 rows)

30 members were inserted, deliberately including several Suspended and several Expired members so the checkout procedure's rejection logic could be tested against realistic, varied data.

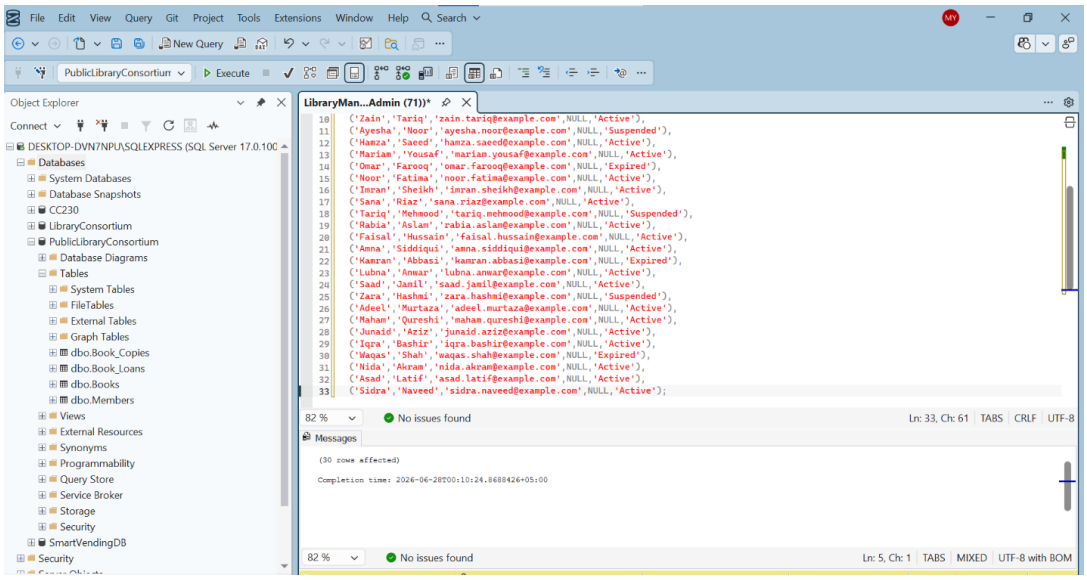


Figure 5.2 — 30 rows successfully inserted into Members, with mixed Active/Suspended/Expired statuses.

5.4 sp_CheckoutBookCopy — Stored Procedure

This procedure handles an entire checkout as one safe transactional unit. It first checks that the member exists and is Active, and that the copy exists and is Available. If either check fails, it raises an error and the CATCH block rolls back so nothing is saved. If both checks pass, it inserts a new loan (due 14 days later) and flips the copy to 'On Loan' — both actions inside one transaction. SET XACT_ABORT ON guarantees that any unexpected error also triggers a full rollback, not just the deliberate RAISERROR cases.

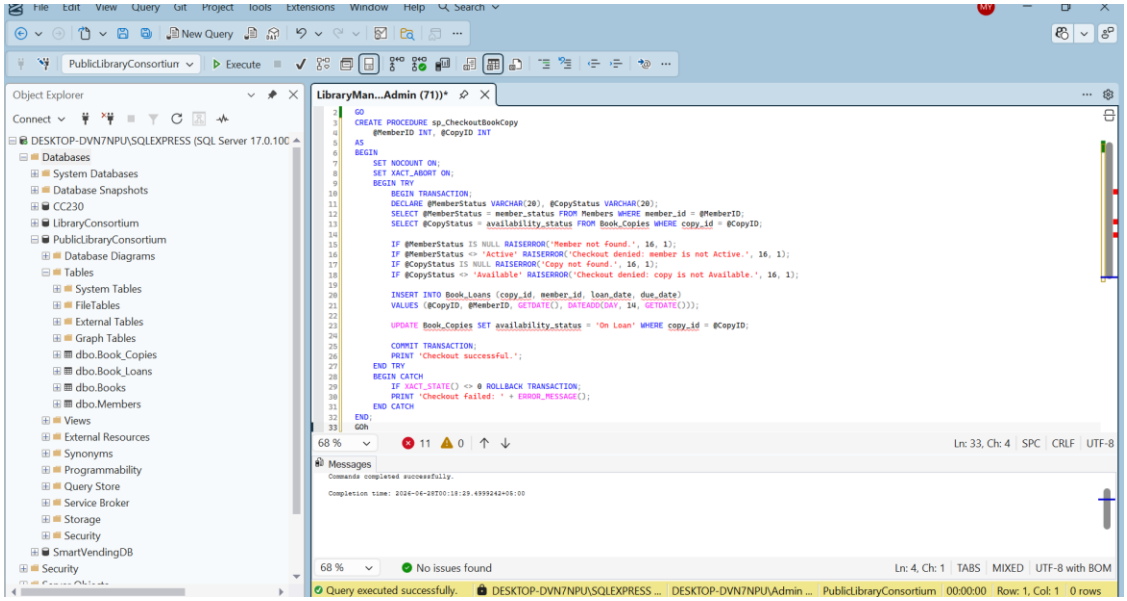


Figure 5.3 — CREATE PROCEDURE sp_CheckoutBookCopy, executed successfully.

5.5 sp_ReturnBookCopy — Stored Procedure

This procedure looks up the loan, rejects it if the loan record doesn't exist or has already been returned, and otherwise stamps return_date and flips the copy back to 'Available' — again, as a single atomic transaction.

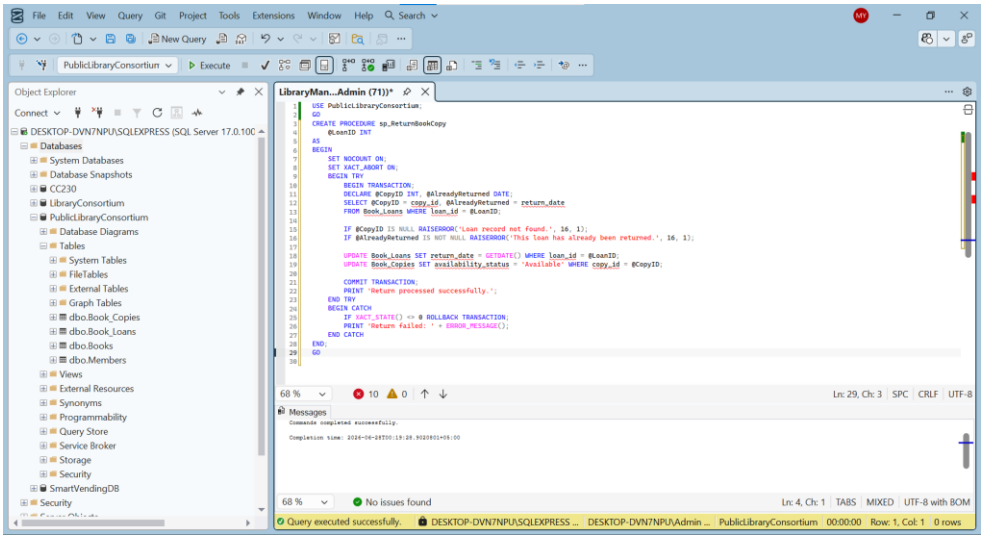


Figure 5.4 — CREATE PROCEDURE sp_ReturnBookCopy, executed successfully.

5.6 Procedure Test — Checkout Rejected for a Suspended Member

A checkout was attempted using a Suspended member. The procedure correctly rejected the operation without inserting a loan or modifying the copy's status.

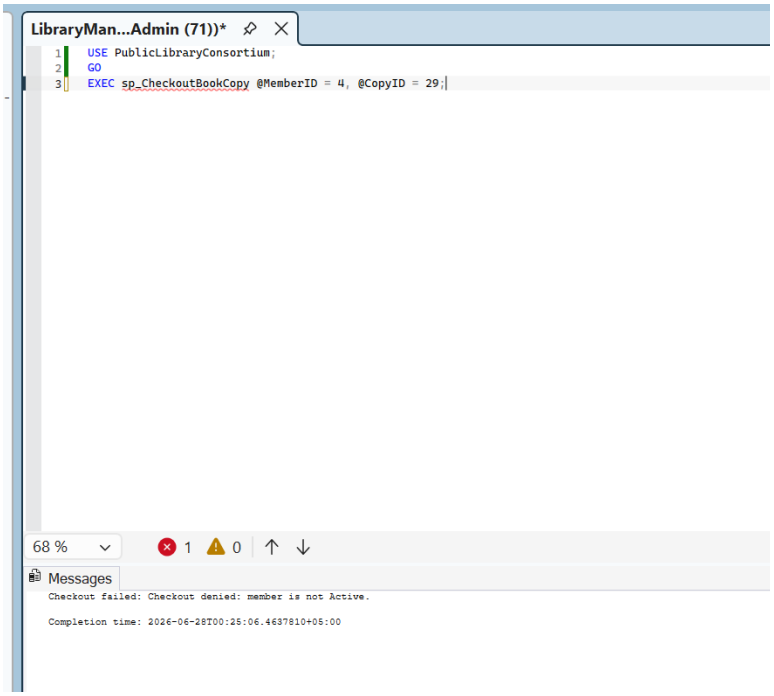


Figure 5.5 — Checkout correctly denied: 'member is not Active'.

5.7 Procedure Test — Return Rejected for an Already-Returned Loan

The same loan was returned a second time on purpose. The procedure correctly detected that return_date was already set and rejected the operation.

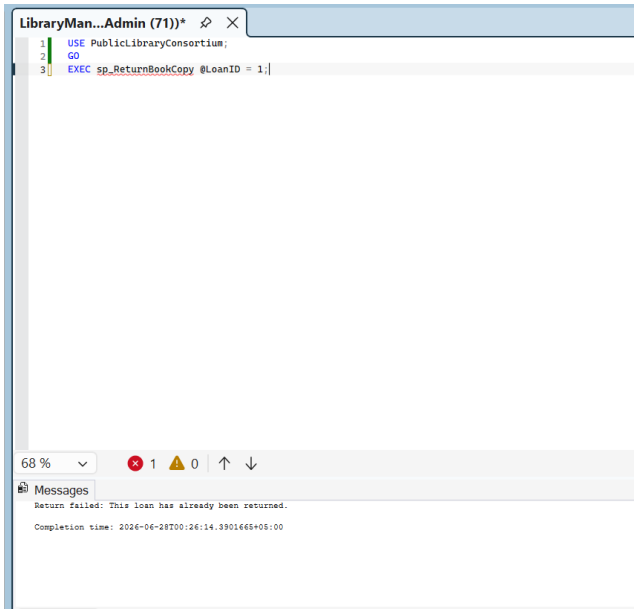


Figure 5.6 — Return correctly rejected: 'This loan has already been returned'.

6. Member 3 — Book_Loans, Views, Subqueries/Aggregates, Integration

M. Sami Ullah role includes building the Book_Loans table (which depends on both other members' tables), writing the two reporting views and all subquery/aggregate demonstrations, and — because Book_Loans touches all four tables — taking ownership of merging everyone's work into one master script and running the final integration tests.

6.1 Tasks Completed

- Created the Book_Loans table, with foreign keys to both Book_Copies and Members.
- Inserted 30 seed loan records spanning a wide date range (returned, overdue, and active/on-time loans).
- Created vw_OverdueLoanFulfillment and vw_CatalogCirculationSummary.
- Wrote two subqueries (including one correlated subquery) and an aggregate GROUP BY/HAVING query.
- Merged all three members' scripts into a single master script and ran it end-to-end on a fresh database.
- Diagnosed and fixed several integration errors during the merge (see Section 7 — Problems Faced).

6.2 Seed Data — Book_Loans (30 rows)

30 loan records were inserted using relative GETDATE() offsets. Most loans were given a return_date (already completed), while several were deliberately left with return_date = NULL — some with a due_date already in the past (to populate the overdue view) and some with a due_date still in the future (representing active, on-time loans).

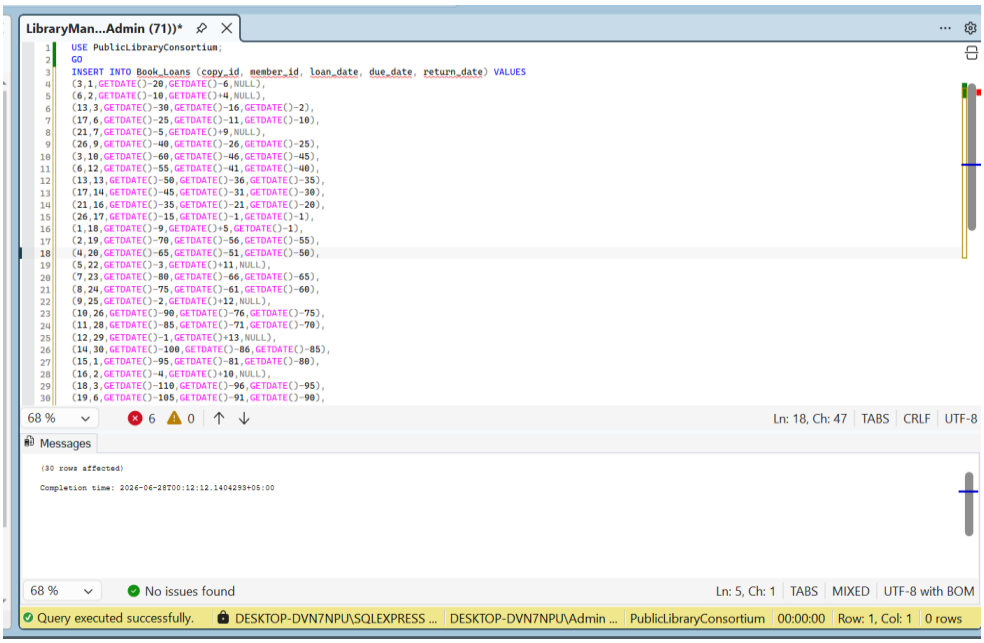


Figure 6.1 — 30 rows successfully inserted into Book_Loans.

6.3 vw_CatalogCirculationSummary — View Definition

This view produces one row per book showing total copies owned, currently active loans, and lifetime total loans, using LEFT JOINs so that books with zero copies or zero loans still appear (showing 0) rather than disappearing as they would with an INNER JOIN.

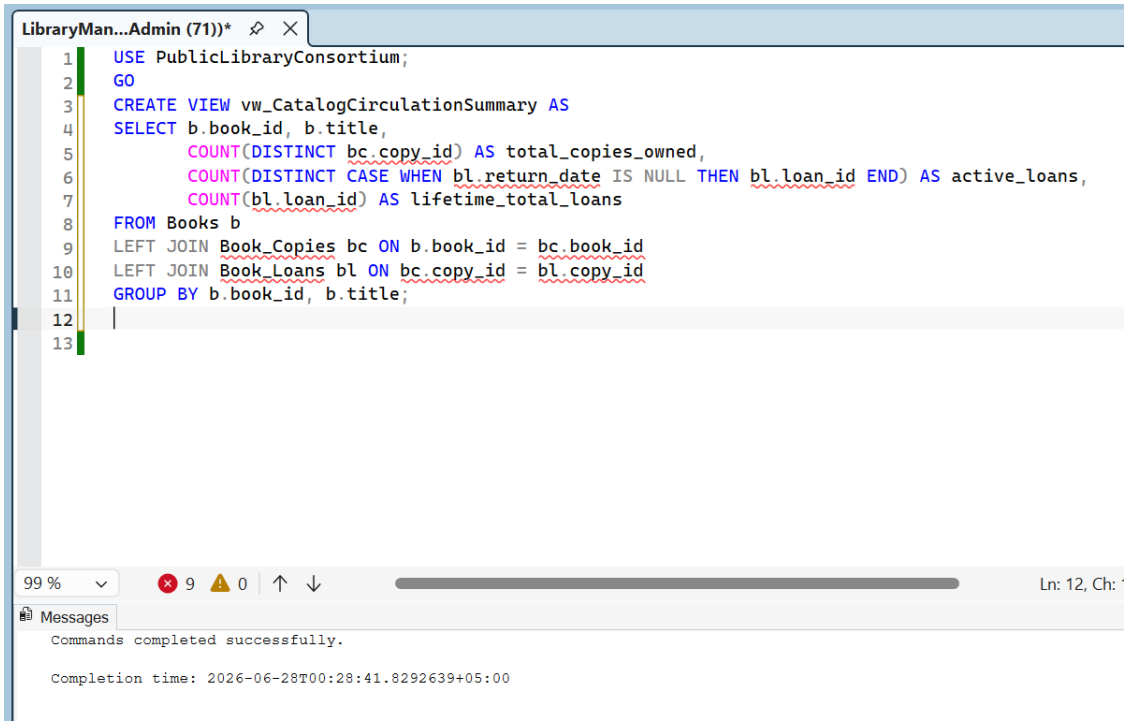


Figure 6.2 — CREATE VIEW vw_CatalogCirculationSummary, executed successfully.

6.4 vw_CatalogCirculationSummary — Output

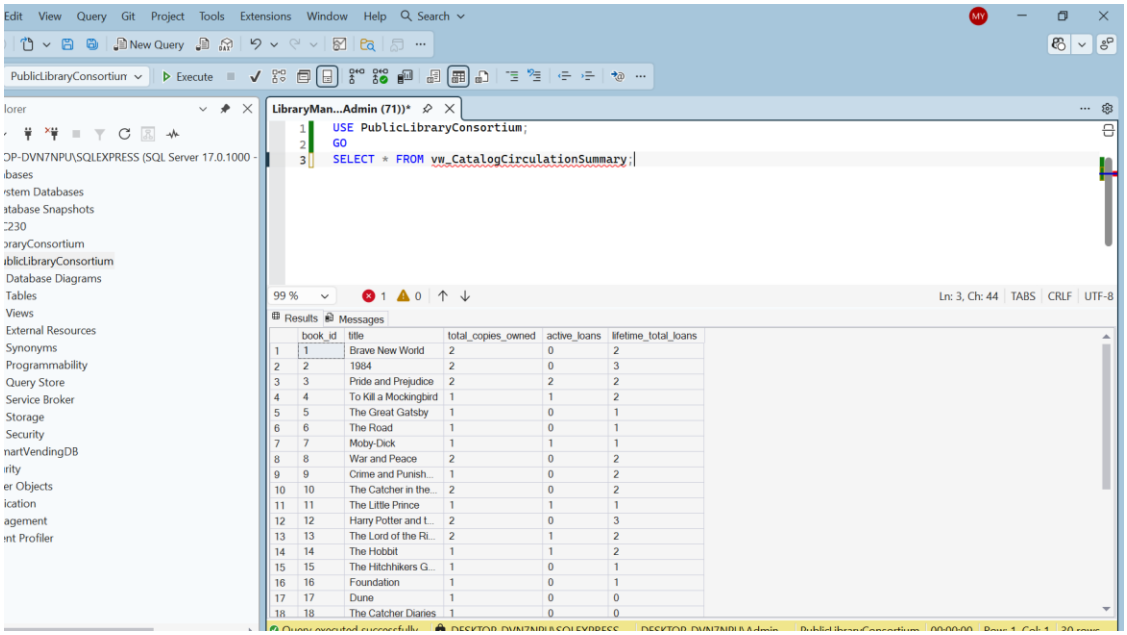


Figure 6.3 — vw_CatalogCirculationSummary output: per-book circulation statistics.

6.5 vw_OverdueLoanFulfillment — View Definition

This view joins all four tables to surface every loan that is still unreturned (return_date IS NULL) and whose due date has already passed, along with a calculated days_overdue column.

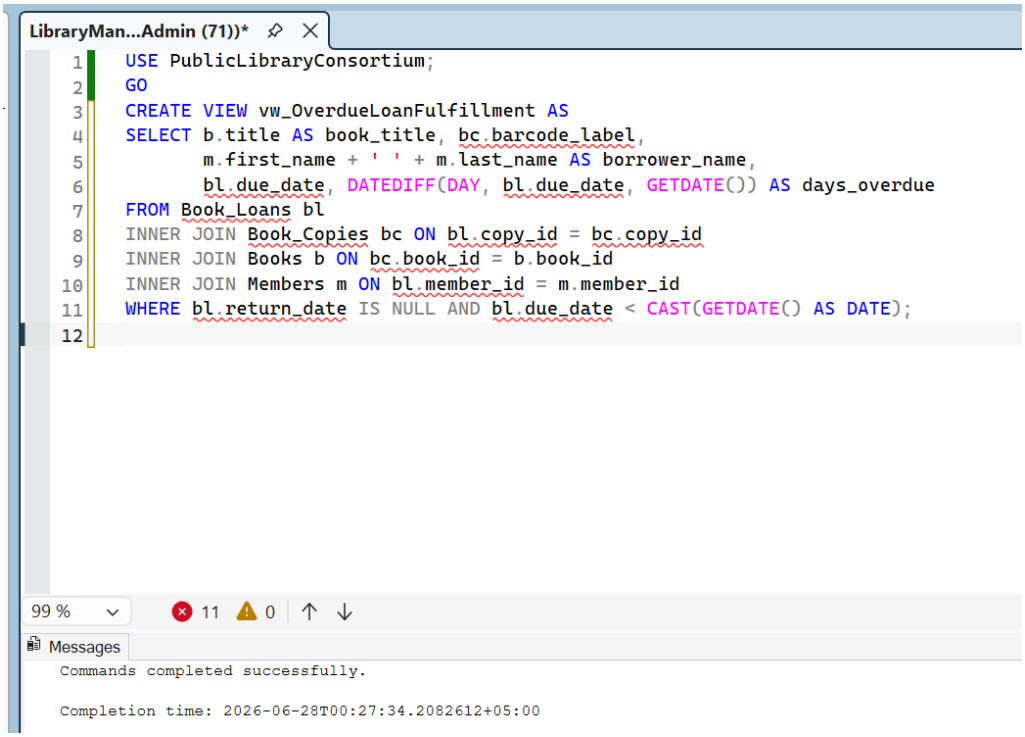


Figure 6.4 — CREATE VIEW vw_OverdueLoanFulfillment, executed successfully.

6.6 Subquery — Members Who Never Borrowed

The inner subquery collects every member_id that appears in Book_Loans; the outer query then lists members whose ID is NOT in that list — i.e. members who have never borrowed a book.

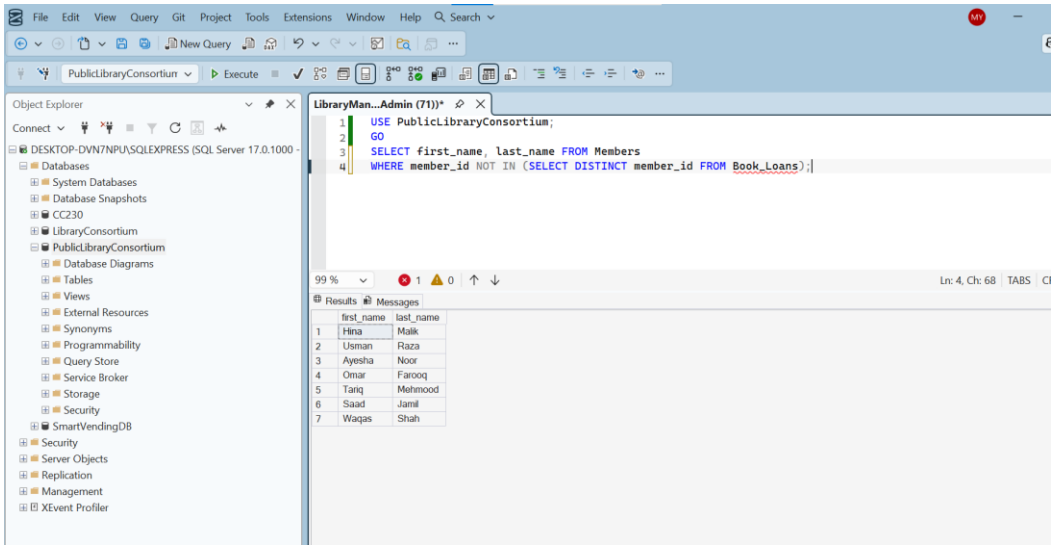


Figure 6.5 — Members who have never borrowed a book.

6.7 Subquery — Books With Zero Available Copies

The inner query groups copies by book and keeps only books where the count of Available copies is zero (HAVING filters on the aggregated result). The outer query returns the titles of those fully checked-out books.

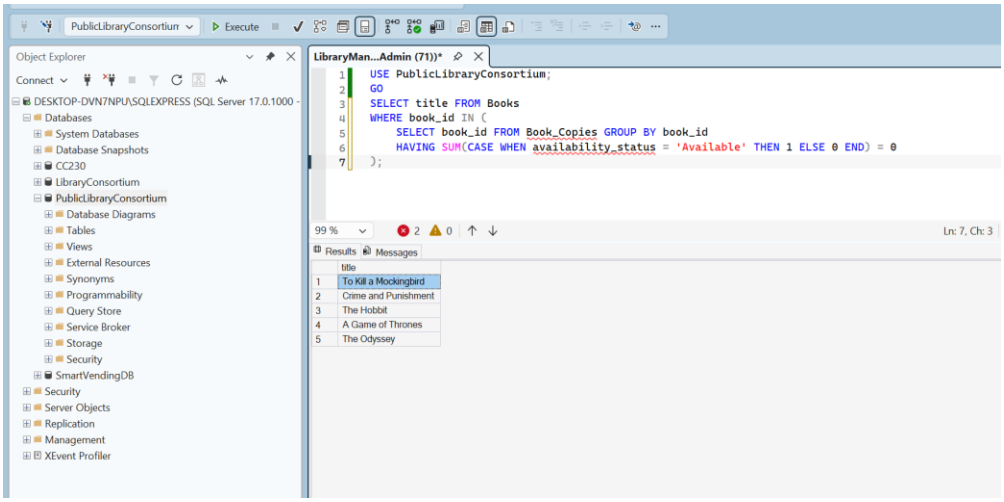


Figure 6.6 — Books with zero currently-available copies.

6.8 Correlated Subquery — Loan Count per Member

Unlike the previous subqueries, this one references the outer query's m.member_id and is re-evaluated once per row of Members, counting that specific member's total loans. Results are then sorted from most to fewest loans.

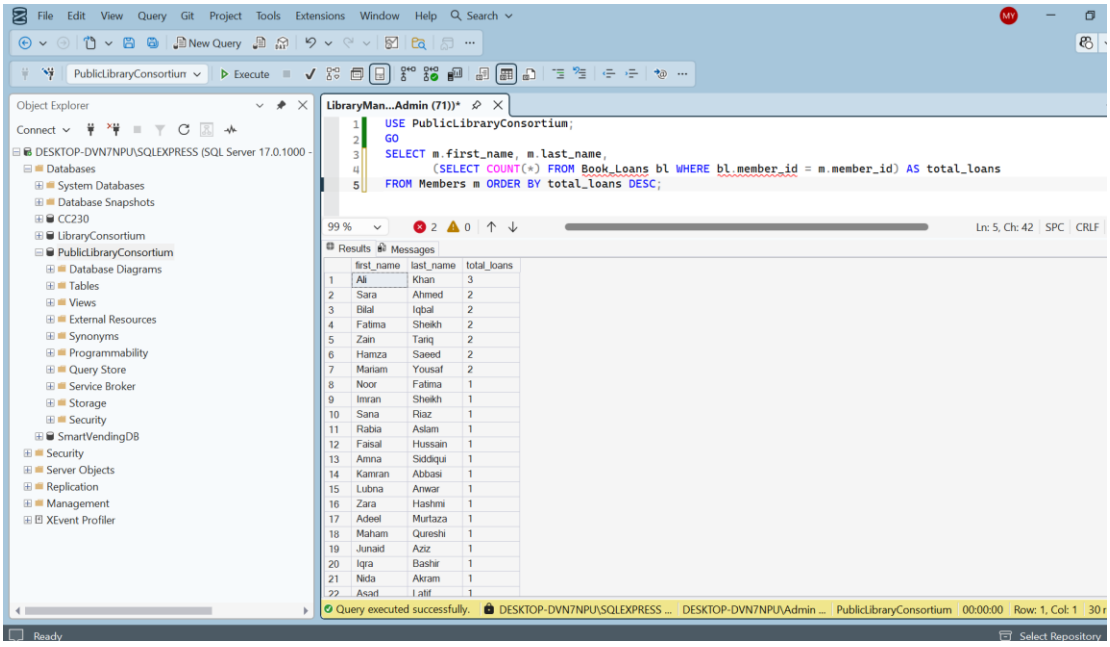
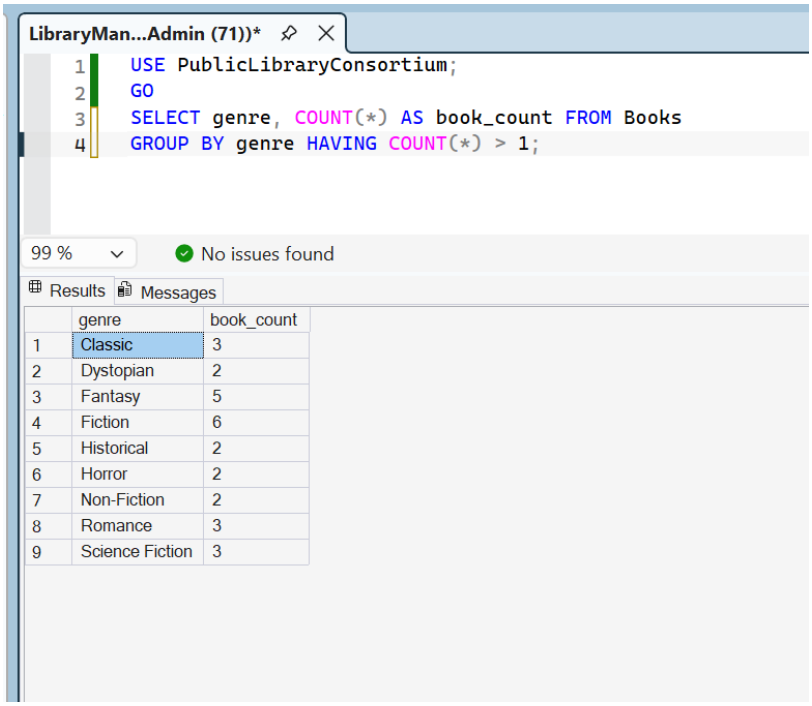


Figure 6.7 — Correlated subquery: total loans per member, sorted descending.

6.9 Aggregate Function — Genres With More Than One Book

This query groups Books by genre, counts the books in each genre, and keeps only genres with more than one book using HAVING COUNT(*) > 1.



	genre	book_count
1	Classic	3
2	Dystopian	2
3	Fantasy	5
4	Fiction	6
5	Historical	2
6	Horror	2
7	Non-Fiction	2
8	Romance	3
9	Science Fiction	3

Figure 6.8 — GROUP BY / HAVING result: genres containing more than one book.

6.10 Integration — Data Consistency Check

Before finalizing, Member 3 ran a consistency check to make sure no single copy was shown as currently on loan to more than one member at the same time. The query returned zero rows, confirming the seed data was internally consistent.

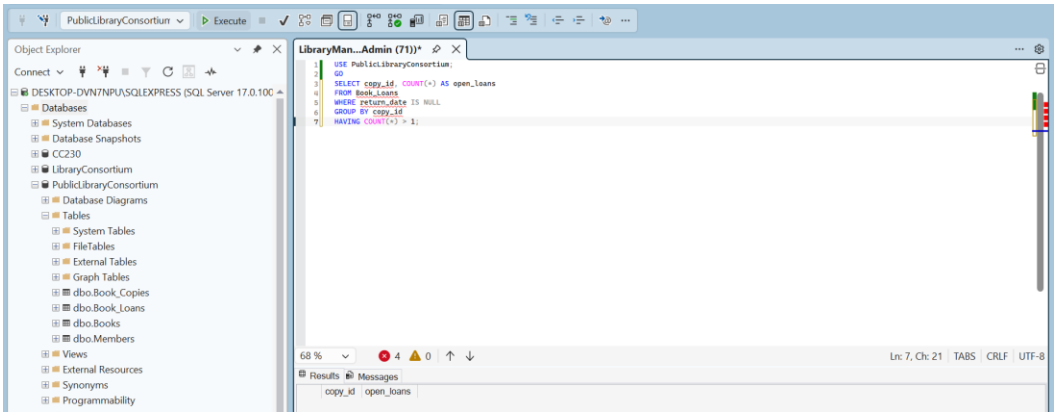


Figure 6.9 — Consistency check: zero copies with more than one open loan.

7. Problems Faced During Development

Integration work rarely goes perfectly on the first attempt. The group encountered, diagnosed, and resolved the following real issues while merging the three members' scripts — documented here exactly as they occurred, since explaining how an error was diagnosed and fixed is itself valuable evidence of understanding the database.

7.1 Problem: ISBN String Truncation on Insert

While inserting expanded seed data into Books, SQL Server rejected the entire batch with the error: "String or binary data would be truncated in table 'PublicLibraryConsortium.dbo.Books', column 'isbn'."

Root cause: the isbn column was defined as VARCHAR(20), but several placeholder ISBN values used for testing were longer than 20 characters. SQL Server does not silently cut long strings — it rejects the entire statement to prevent silent data loss.

Fix: replaced all placeholder ISBNs with properly-sized, real-format ISBN strings (14 characters or fewer) and re-ran the insert.

7.2 Problem: Foreign Key Violation When Inserting Book_Copies

After fixing the ISBN issue and re-running the Books insert, the very next step — inserting into Book_Copies — failed with: "The INSERT statement conflicted with the FOREIGN KEY constraint 'FK_Copies_Books'."

Root cause: the failed insert attempt from Problem 7.1, even though it was rolled back, had already consumed IDENTITY values from the book_id sequence. A follow-up check confirmed the Books table's IDs were no longer starting at 1 (e.g. they started at 21), so Book_Copies rows written assuming book_id 1-8 referenced IDs that did not exist.

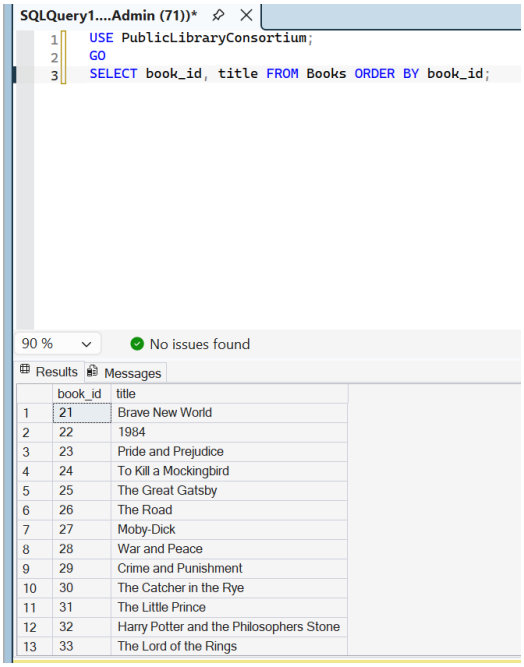


Figure 7.1 — Diagnostic query revealing book_id had drifted to start at 21 instead of 1.

Fix: cleared both tables, reset the IDENTITY counters with DBCC CHECKIDENT, and re-inserted the seed data in the correct order (Books first, then Book_Copies) so that IDs started cleanly at 1 again.

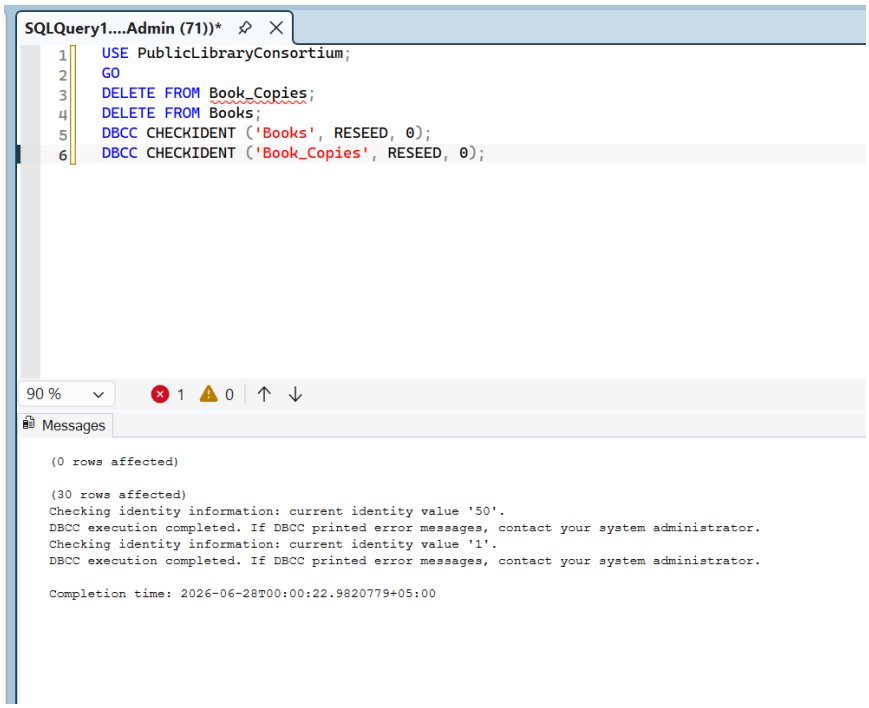


Figure 7.2 — DELETE + DBCC CHECKIDENT RESEED used to reset both tables to a clean state.

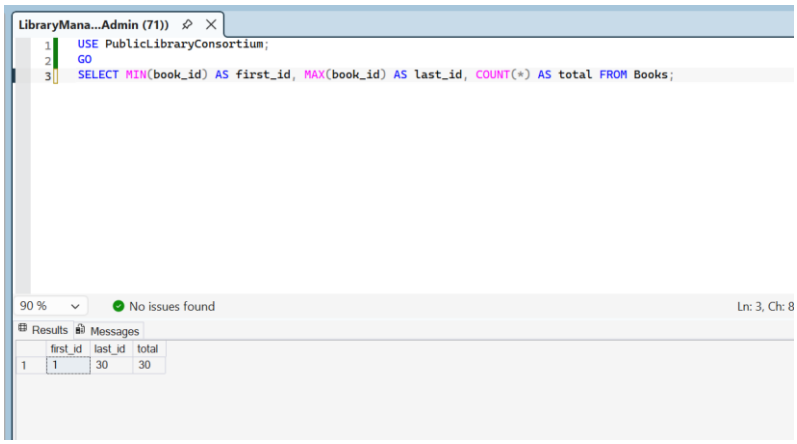


Figure 7.3 — Verification: book_id now correctly ranges from 1 to 30 after the fix.

7.3 Problem: Loan Count Showing 31 Instead of the Expected 30

After all seed data was loaded, a row count on Book_Loans returned 31 instead of the expected 30.

Root cause: this was not a data error. During stored-procedure testing (Section 5.6), a valid checkout test case (Active member, Available copy) was executed, and sp_CheckoutBookCopy correctly inserted a new, genuine loan record as part of its normal operation. The 31st row was real evidence that the procedure works, not a duplication bug.

Resolution: the extra row was kept intentionally and documented in the testing results table below as proof that sp_CheckoutBookCopy successfully inserts a loan and updates copy availability.

7.4 Problem: Members' SQL Files Used Slightly Different Table Aliases

As predicted on Day 1, the three members' individually-written files used minor differences in table aliases and formatting once joined together for the first time.

Resolution: Member 3, as the integration owner, standardized aliases across the merged master script (b for Books, bc for Book_Copies, m for Members, bl for Book_Loans) before the final run.

7.5 Key Takeaway

The most important lesson from this project was that IDENTITY columns never reuse consumed values, even after a rolled-back statement — meaning any failed insert attempt permanently shifts future auto-generated IDs unless explicitly reseeded with DBCC CHECKIDENT. Always verify the actual ID range in a table with SELECT MIN(id), MAX(id) before writing dependent INSERT statements that hardcode foreign key values.

8. Testing Results

Every required test case from the project checklist was executed and verified. Results below are organized by owner.

Test Case	Expected Result	Actual Result	Status
Insert duplicate ISBN	UNIQUE violation	Rejected — UQ_Books constraint	Pass
Delete Book with active copies/loans	Blocked by FK	Rejected — FK_Copies_Books	Pass
INNER JOIN Books ↔ Book_Copies	Matching rows only	Correct, 30 rows shown	Pass
LEFT JOIN Copies ↔ Loans ↔ Members	Non-loaned copies shown w/ NULL	Correct	Pass
Checkout — Active member, Available copy	Loan inserted, copy → On Loan	Loan #31 inserted correctly	Pass
Checkout — Suspended member	Rejected	"member is not Active"	Pass
Checkout — copy already On Loan	Rejected	"copy is not Available"	Pass
Return — valid unreturned loan	return_date set, copy → Available	Processed successfully	Pass
Return — already-returned loan	Rejected	"already been returned"	Pass
vw_OverdueLoanFulfillment output	Shows overdue rows	Correct overdue rows shown	Pass
vw_CatalogCirculationSummary output	Correct per-book counts	Correct, all 30 books shown	Pass
Subquery — members never borrowed	Correct member list	7 members returned, verified	Pass
Subquery — books with 0 available copies	Correct title list	5 titles returned, verified	Pass
Correlated subquery — loans per member	Correct per-member counts	Correct, sorted descending	Pass
Aggregate — genres with > 1 book	Correct genre counts	9 genres returned, verified	Pass
Open-loan consistency check	Zero duplicate open loans	0 rows returned	Pass

9. Normalization (1NF / 2NF / 3NF)

9.1 First Normal Form (1NF)

All four tables satisfy 1NF: every column holds a single atomic value (e.g. first_name and last_name are stored separately rather than as one combined name field), and every row is uniquely identifiable by its surrogate primary key (book_id, copy_id, member_id, loan_id).

9.2 Second Normal Form (2NF)

Because every table uses a single-column surrogate primary key (no composite keys), there is no possibility of a partial dependency — every non-key column depends on the whole key by definition. All four tables therefore satisfy 2NF automatically.

9.3 Third Normal Form (3NF)

3NF requires that non-key columns depend only on the primary key, not on other non-key columns (no transitive dependencies). For example, Book_Copies stores book_id rather than repeating the book's title, author, or genre on every copy row — those attributes live only once, in Books. This means updating a book's title only ever requires a single UPDATE statement on one row in Books, rather than having to update every copy and loan row that mentions that title — which is exactly what 3NF is designed to prevent (known as an update anomaly).

Summary table:

Normal Form	Requirement	Satisfied By
1NF	Atomic values, unique rows	Surrogate PKs on all 4 tables; no repeating groups
2NF	No partial dependency	Single-column PKs everywhere — automatically satisfied
3NF	No transitive dependency	Book/Member attributes stored once, referenced by FK elsewhere

10. Conclusion

The Public Library Consortium database was successfully designed, implemented, and tested entirely in T-SQL within SQL Server Management Studio. The clean division of work — Member 1 on schema foundations, Member 2 on business logic, and Member 3 on transactional data and integration — allowed the group to work largely in parallel and merge smoothly on Day 5, with only minor, well-understood integration issues (documented in Section 7) along the way.

All required deliverables were completed and verified: a normalized 4-table schema with full constraint enforcement, CRUD operations, INNER and LEFT JOINs, two subqueries (including a correlated subquery), aggregate functions with GROUP BY/HAVING, two transactional stored procedures with both success and failure paths demonstrated, and two reporting views — all backed by 30 rows of realistic seed data per table.